Computer Graphics Using OpenGL
INM376 / IN3005

Module Leader – Dr Eddie Edwards

Lecture 09: Intermediate / Advanced Graphics 2

MSc Games Tech – Drinks & Snacks

Social event – all welcome

The Old Ivy House, 6pm 1st April

Recap of last time

- Advanced texture mapping
  - Environment mapping: Reflection / refraction
  - Multi-texturing
  - Other applications (light mapping, bump mapping, etc.)
- Fog
- Shadows

Overview of today's lecture

- Geometry shaders
- Computer animation
  - Framerate considerations
  - Keyframing
  - Skinning
  - Procedural methods

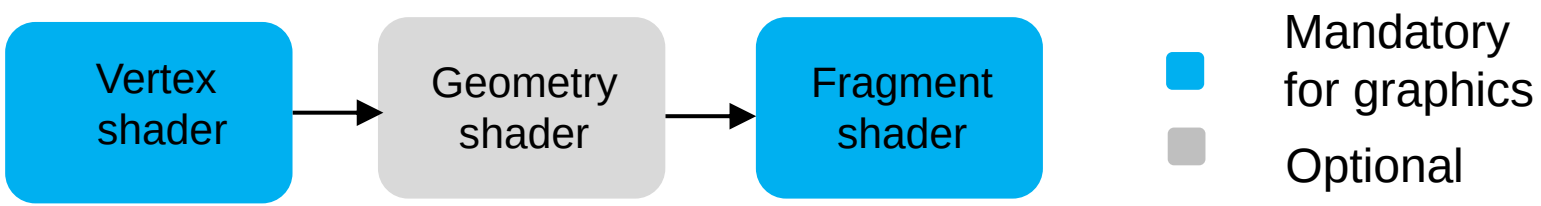
Questions

- Help! My program doesn't run properly in Release (Debug) mode!
- When a program works in Debug mode, but not Release mode, there is usually some sort of memory error, or error with OpenGL
- Solution: use `glGetError()` to try to determine if/where an error is occurring
  - For example, put `GLenum e = glGetError();` in your Game::Render() method.
  - If an error is generated, work back (as discussed in Lecture 2) to find what is causing the error.

- Examples
  - Binding to an unallocated texture causes error GL_INVALID_OPERATION (1282)
  - Setting uniforms in programs that are not being used (through glUseProgram) causes error GL_INVALID_OPERATION (1282)

Geometry shaders

- Between the vertex and fragment shader, the geometry shader is another shader type available to the OpenGL programmer.
- As mentioned before, its use is optional. When a geometry shader is present, the outputs of the vertex shader go to the geometry shader. The outputs of the geometry shader are then sent to the fragment shader.



- Vertex shaders process one *vertex* at a time
- Geometry shaders process one *primitive* at a time
- Fragment shaders process one *fragment* at a time

Geometry shaders

- Vertex shaders cannot access any other vertex's information and is strictly one in, one out. A vertex shader cannot create new vertices and cannot discard the vertex.
- Likewise, fragment shaders cannot access any other fragment's information, cannot create new fragments, but can discard fragments using the *discard* keyword.
- In contrast, geometry shaders have access to **all vertices in the primitive**. They can change the type of primitive, **create** new primitives, and discard primitives as well. However, geometry shaders can only output one primitive *type*.

Geometry shaders

- In the template, to include a geometry shader, just load it from file and make it part of a shader program

```
sShaderFileNames.push_back("myshader.vert");
sShaderFileNames.push_back("myshader.geom");
sShaderFileNames.push_back("myshader.frag");

for (int i = 0; i < (int) sShaderFileNames.size(); i++) {
   ...
   shader.LoadShader("resources\\shaders\\"+sShaderFileNames[i], iShaderType);
   shShaders.push_back(shader);
}
```

```
CShaderProgram *pMainProgram = new CShaderProgram;
pMainProgram->CreateProgram();
pMainProgram->AddShaderToProgram(&shShaders[0]);
pMainProgram->AddShaderToProgram(&shShaders[1]);
pMainProgram->AddShaderToProgram(&shShaders[2]);
pMainProgram->LinkProgram();
m_pShaderPrograms->push_back(pMainProgram);
```

A simple pass-through geometry shader

- Let's consider a simple pass-through geometry shader, which doesn't actually do anything other than pass through data, *from* the vertex shader *to* the fragment shader.

vertex shader

```
...
out vec3 vColourPass;
out vec2 vTexCoordPass;
void main()
{
   gl_Position = matrices.projMatrix * matrices.modelViewMatrix
                 * vec4(inPosition, 1.0f);
   vec3 n = normalize(matrices.normalMatrix * inNormal);
   vec4 p = matrices.modelViewMatrix * vec4(inPosition, 1.0f);
   vColourPass = PhongModel(p, n);
   vTexCoordPass = inCoord;
}
```

A simple pass-through geometry shader

geometry shader

```
#version 400
in vec3 vColourPass[];
in vec2 vTexCoordPass[];
out vec3 vColour;
out vec2 vTexCoord;
layout(triangles) in;
layout(triangle_strip, max_vertices = 3) out;

void main() {
   for(int i = 0; i < 3; i++) {
      gl_Position = gl_in[i].gl_Position;
      vColour = vColourPass[i];
      vTexCoord = vTexCoordPass[i];
      EmitVertex();
   }
   EndPrimitive();
}
```

A simple pass-through geometry shader

- First two lines of top part:

```
#version 400
in vec3 vColourPass[];
in vec2 vTexCoordPass[];
out vec3 vColour;
out vec2 vTexCoord;
layout(triangles) in;
layout(triangle_strip, max_vertices = 3) out;
```

geometry shader

  - `vColourPass[]` and `vTexCoordPass[]` coming in from the vertex shader.
  - Variables that are out for the vertex shader **are arrays** when input to the geometry shader, since the geometry shader will have access to all vertices on the primitive.

A simple pass-through geometry shader

- Rest of the top part:

```
#version 400
in vec3 vColourPass[];
in vec2 vTexCoordPass[];
out vec3 vColour;
out vec2 vTexCoord;
layout(triangles) in;
layout(triangle_strip, max_vertices = 3) out;
```

geometry shader

  - The geometry shader will output a `vColour` and a `vTexCoord` to the fragment shader.
  - The layout of the input primitive is a triangle (of any type).
  - The layout of the output primitive is a triangle strip consisting of a maximum of three vertices. OpenGL uses the maximum number of vertices to allocate internal arrays.

A simple pass-through geometry shader

- Input primitive types include
  - points
  - lines
  - triangles
  - lines_adjacency*
  - triangles_adjacency*
- Output primitive types include
  - points
  - line_strip
  - triangle_strip

* used to access vertices of neighbouring primitives

A simple pass-through geometry shader

- Bottom part:

```
void main() {
   for(int i = 0; i < 3; i++) {
      gl_Position = gl_in[i].gl_Position;
      vColour = vColourPass[i];
      vTexCoord = vTexCoordPass[i];
      EmitVertex();
   }
   EndPrimitive();
}
```

geometry shader

- For each vertex in the primitive, this code:
  - Sets `gl_Position`
  - Passes the colour from the vertex shader
  - Passes the texture coordinate from the vertex shader
  - Emits the vertex. This will complete processing for this vertex
  - Calls `EndPrimitive` to let OpenGL know we're done with this primitive

- Question: How could one discard a primitive using a geometry shader?

Duplication shader

- The pass-through shader shows the basic structure of a geometry shader but it doesn't really do anything exciting.
- Let's modify it to make a duplication shader, which will duplicate an object and translate it by a vector.
- First, let's change the vertex shader so that it passes through the vertex positions. We'll apply the modelview and projection matrices to compute gl_Position *in the geometry shader* instead.

vertex shader

```
...
out vec3 vColourPass;
out vec2 vTexCoordPass;
void main()
{
   gl_Position = vec4(inPosition, 1.0f);
   vec3 n = normalize(matrices.normalMatrix * inNormal);
   vec4 p = matrices.modelViewMatrix * vec4(inPosition, 1.0f);
   vColourPass = PhongModel(p, n);
   vTexCoordPass = inCoord;
}
```

Duplication shader

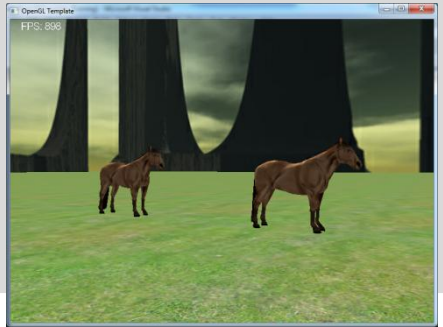
- In the geometry shader, we'll duplicate each triangle and apply a translation:

six vertices

geometry shader

```
...
layout(triangle_strip, max_vertices = 6) out;
void main() {
   for(int i = 0; i < 3; i++) {
      gl_Position = projMatrix * modelViewMatrix * gl_in[i].gl_Position;
      vColour = vColourPass[i];
      vTexCoord = vTexCoordPass[i];
      EmitVertex();
   }
   EndPrimitive();
   vec4 t = vec4(15.0, 0.0, 0.0, 0.0);
   for(int i = 0; i < 3; i++) {
      gl_Position = projMatrix * modelViewMatrix * (gl_in[i].gl_Position + t);
      vColour = vColourPass[i];
      vTexCoord = vTexCoordPass[i];
      EmitVertex();
   }
   EndPrimitive();
}
```

translation vector: w = 0 since gl_in[i].gl_Position.w is already 1



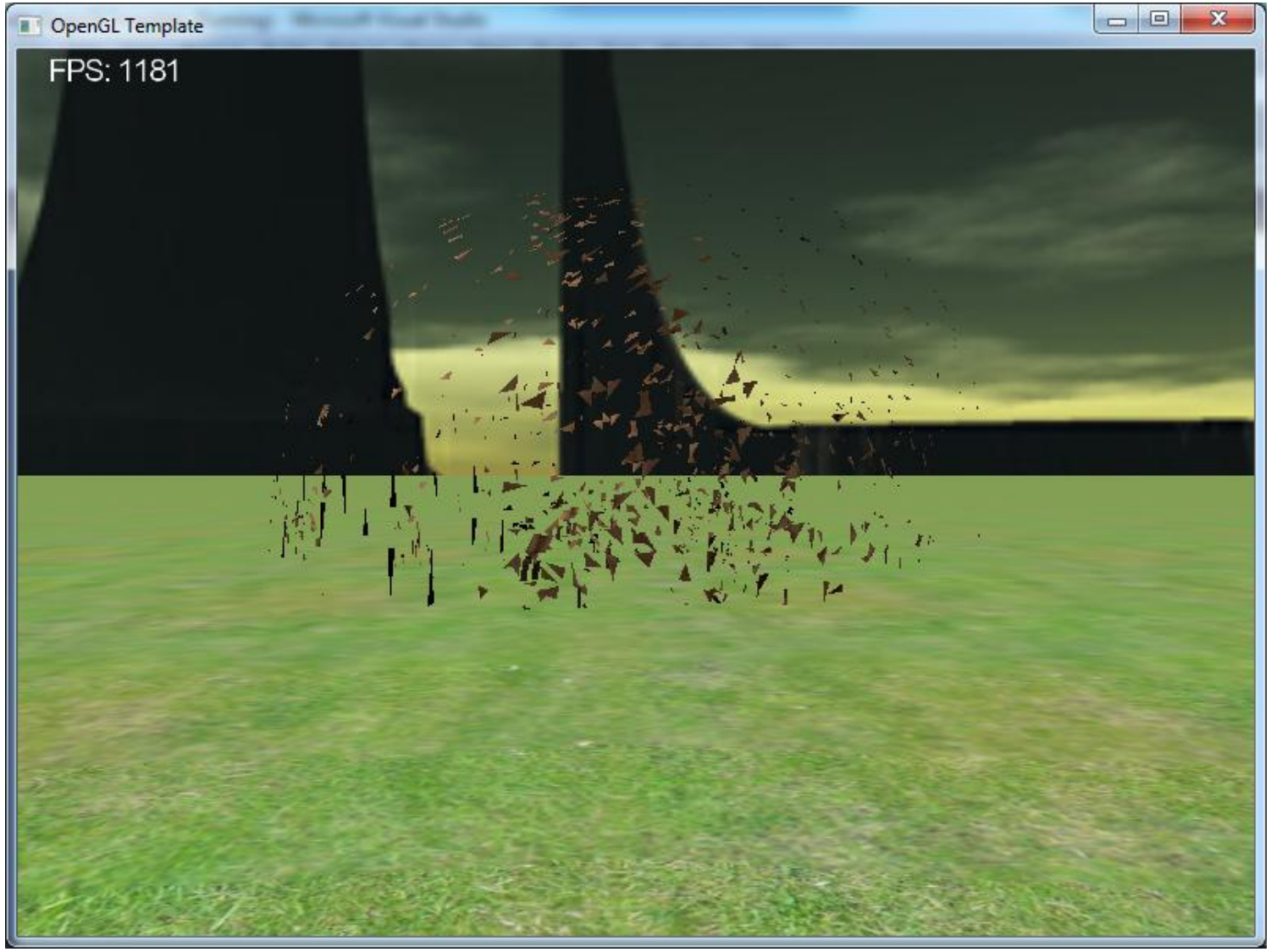
Explosion shader

- We can “explode” an object displacing all triangles along their normal

geometry shader

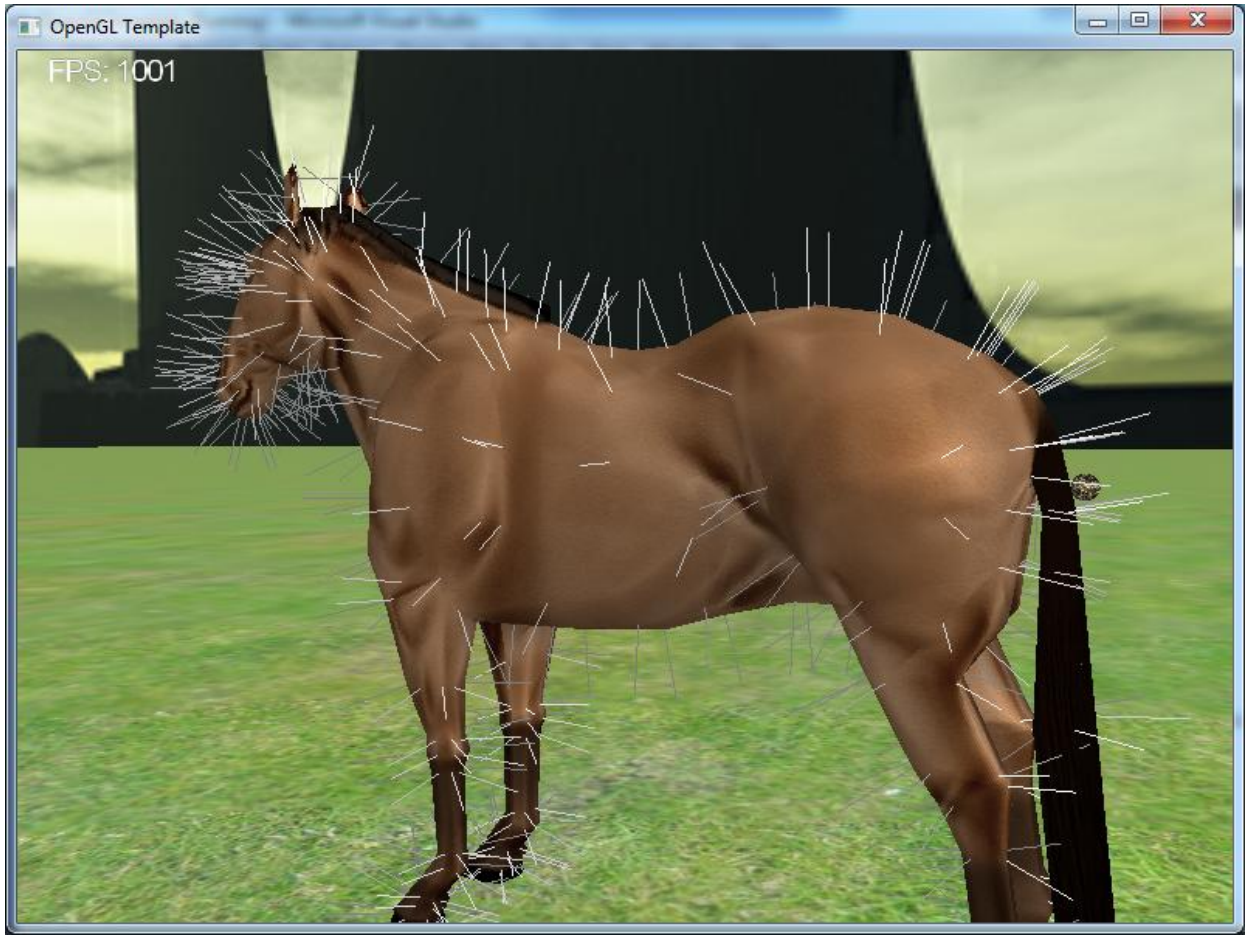
```
uniform bool bExplodeObject;
uniform float explodeFactor;

void main() {
   float localExplode = bExplodeObject? explodeFactor: 0.0;

   // Get triangle normal
   vec3 e1 = gl_in[1].gl_Position.xyz - gl_in[0].gl_Position.xyz;
   vec3 e2 = gl_in[2].gl_Position.xyz - gl_in[0].gl_Position.xyz;
   vec3 n = normalize(cross(e1, e2));
   for (int i = 0; i < 3; i++) {
      vec4 explodedPos = gl_in[i].gl_Position + localExplode * vec4(n, 0);
      gl_Position = projMatrix * modelViewMatrix * explodedPos;
      vColour = vColourPass[i];
      vTexCoord = vTexCoordPass[i];
      EmitVertex();
   }
   EndPrimitive();
}
```

Explosion demo



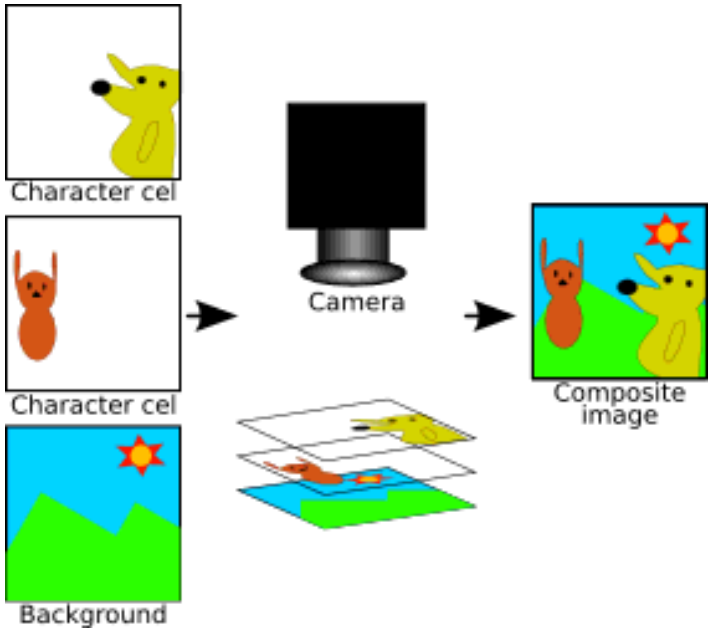
Porcupine rendering

- Horse rendered twice: Once as normal, the second time using a porcupine rendering shader, which draws a line along the normal.



Computer animation

- Computer animation is the digital successor to traditional animation techniques, like
  - Frame-by-frame animation of drawn content
  - Cel animation
  - Stop motion



Types of computer animation

- Keyframing
  - Animator specifies important positions throughout the animation as *keyframes*
  - Keyframes are interpolated to produce intermediate frames – this process is called *tweening*
- Motion capture (mocap)
  - A system captures motion data from a real enactment of the animation
  - The data is then transferred to a digital model to control a virtual character
- Procedural
  - A set of equations or rules are evaluated to animate a object or scene
  - This includes physically-based animation techniques

Keyframing

- Keyframing is based on the idea that an animated object has a beginning state that changes over time, reaching a final state.
- The state may include position, shape, colour, or other properties.
- In keyframing, the animator only specifies the “key” frames, and the intermediate frames between the start and end states are found through interpolation.
- In traditional animation, often keyframes were specified by a senior artist and the tweened frames were created manually by less experienced (and lower paid) artists; today it is typically done using interpolation on a computer.

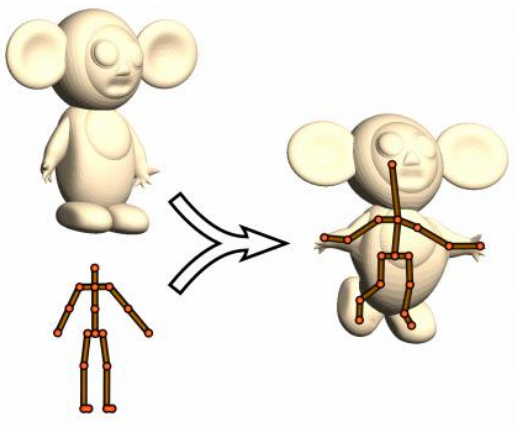
MD2 file rendered using keyframe interpolation

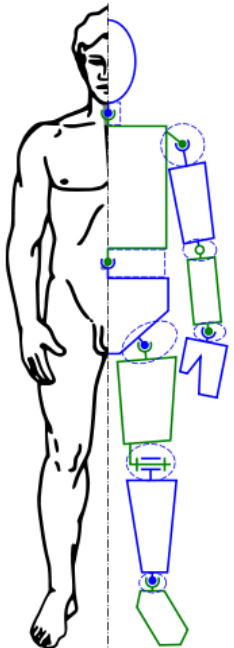
Example

https://www.youtube.com/watch?v=f3Cr8Yx3GGA

Skeletal animation

- Modeling the motion of each individual vertex on a mesh for each keyframe can be tedious as there are many degrees of freedom.
- Instead, an articulated character model is often described by a set of body parts and joint angles in a kinematic chain.
  - The body parts are often called *bones*
  - The joint angles allow control through a reduced set of variables (usually 20 – 25 bones for a human character)
- This type of modeling is called skeletal animation (or bone animation), and is supported by the MD3/MD5 format.

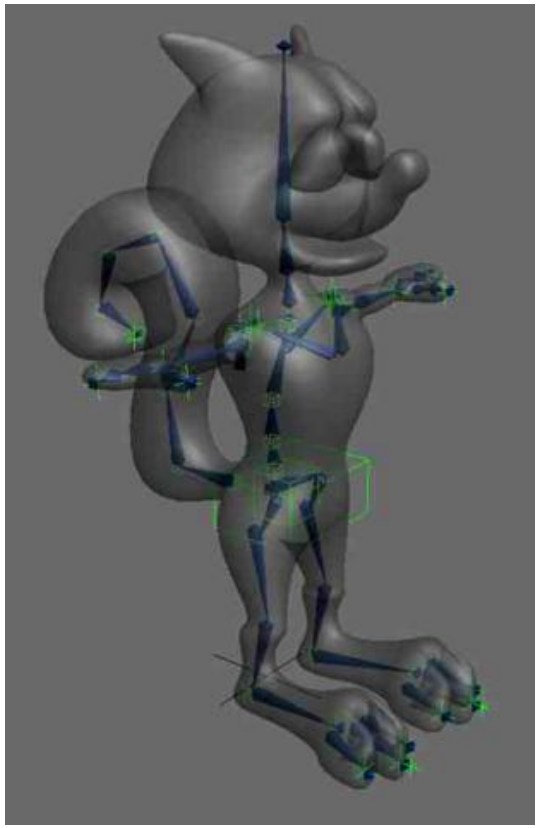




http://people.csail.mit.edu/ibaran/papers/2007-SIGGRAPH-Pinocchio.pdf

Rigging

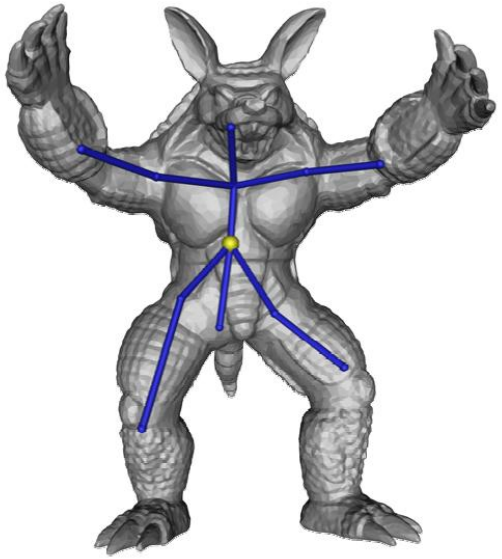
- Building the animation controls is called *rigging*. In a skeletal animation, rigging involves defining the skeleton and associating it to the mesh.
- Bones in the skeleton are moved by an animator.
- Surface vertices associated with a bone are moved when a bone moves.
- The bones provide a simple way for the animator to set the character's pose in different keyframes.



Steve Marschner

Skinning

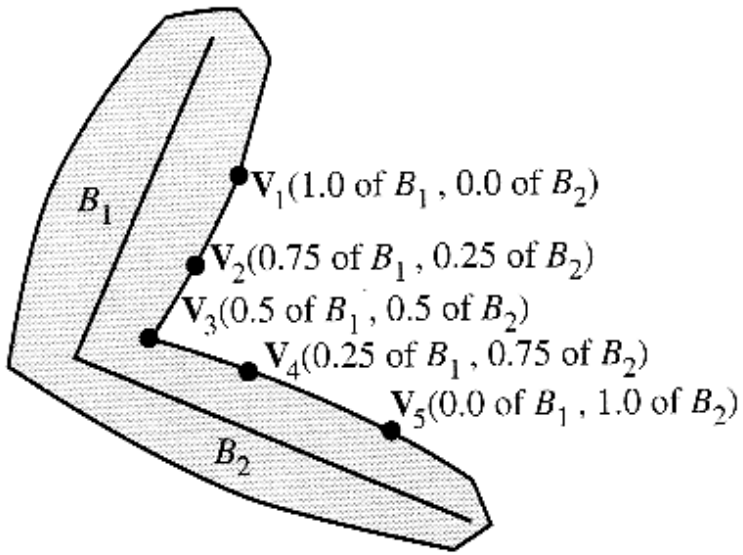
- The skeleton may be updated in a number of ways such as changing the joint angles at different keyframes, which can be interpolated over time.
- When the skeleton is repositioned, the mesh must then be updated.
- In *skinning*, the mesh vertices are considered as the “skin” around the bones forming the skeleton. The mesh is deformed smoothly as the skeleton moves.
- Skinning is also known as *skeletal subspace deformation*.

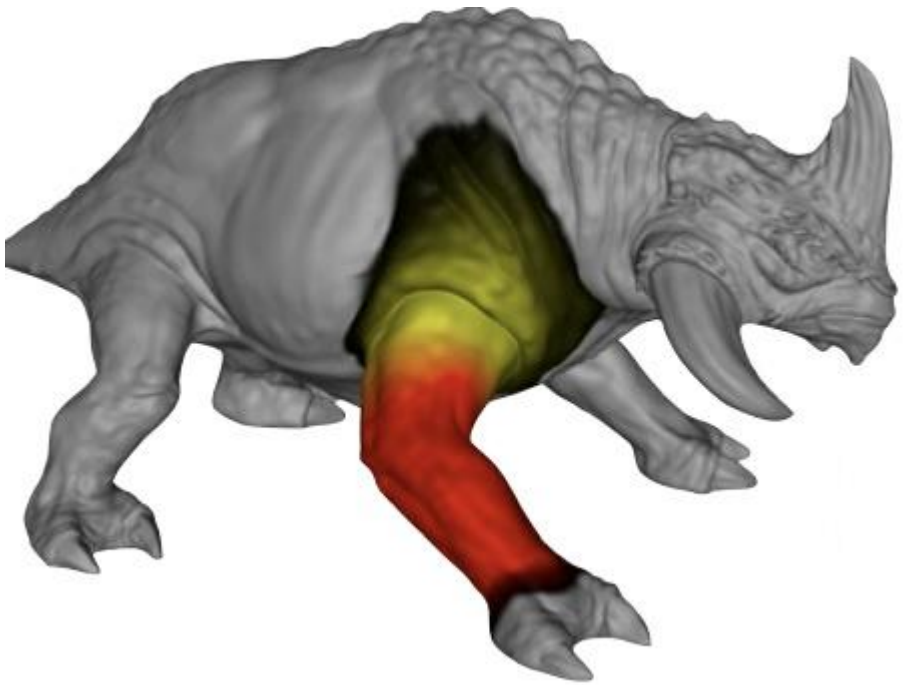


http://faculty.cs.tamu.edu/schaefer/research/slides/skeletonization.pdf

Example

- Vertices are associated with bones. Near joints, the vertices are associated with multiple bones coming into the joint.





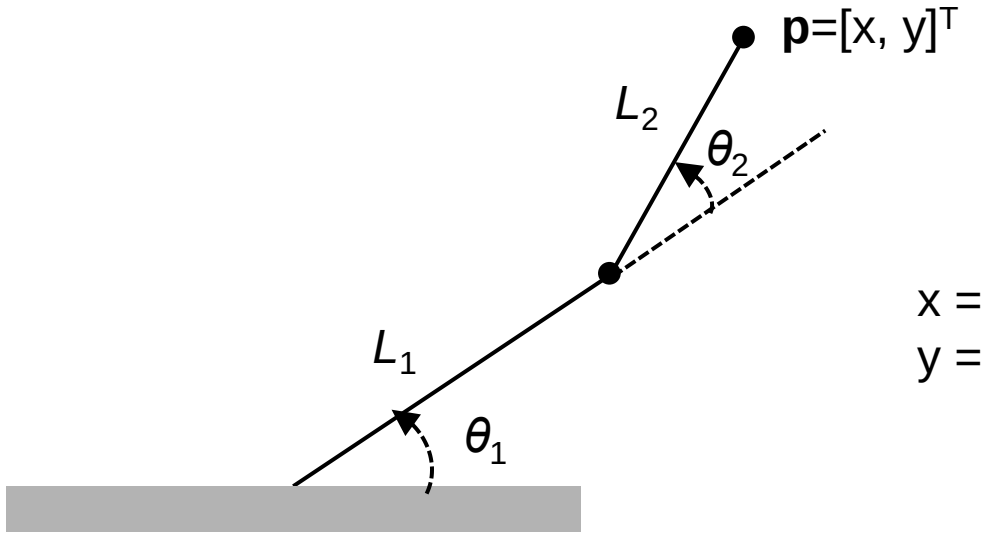
Wang and Philips 2002

Example

https://www.youtube.com/watch?time_continue=22&v=psut1SvSrUQ
https://www.youtube.com/watch?v=r6Yv_mh79PI

Forward kinematics

- In **forward kinematics**, an animator will specify the joint angles of the skeleton.
- Then, the bone positions are computed from the joint angles.
- Example: given θ_1 and θ_2, determine the location of **p**

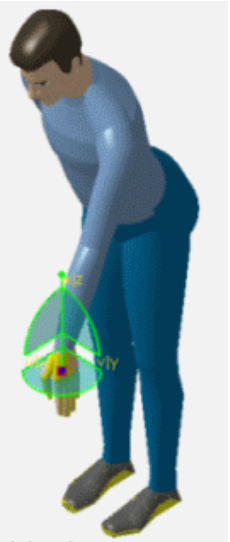


$$x = L_1 \cos\theta_1 + L_2 \cos(\theta_1+\theta_2)$$
$$y = L_1 \sin\theta_1 + L_2 \sin(\theta_1+\theta_2)$$

⇨ Animating joint angles like this might not be intuitive for an animator

Inverse kinematics

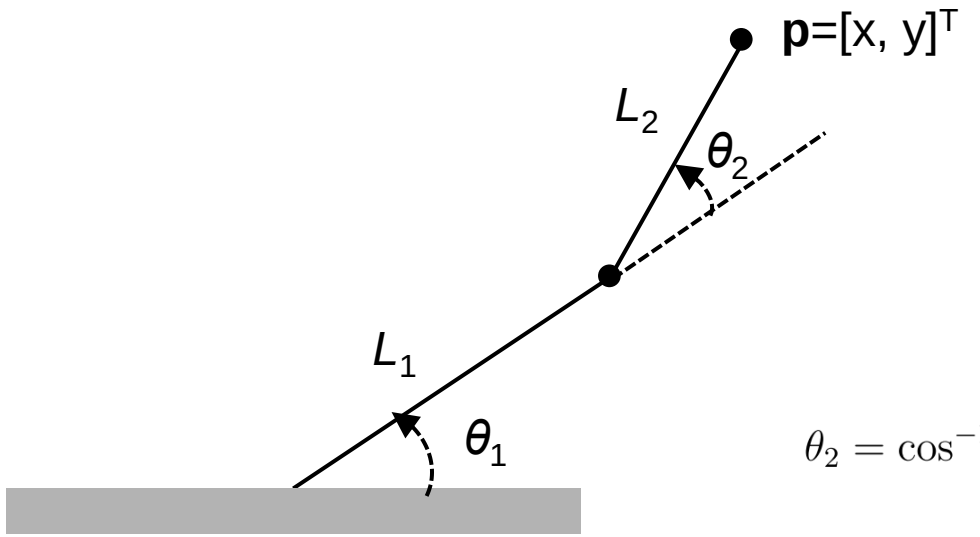
- In **inverse kinematics**, an animator will specify where the object should be located. For example, positioning the character in a particular pose.
- A system of equations is then solved to determine the joint angles.
- Example: inverse kinematics allows an artist to move the hand of a 3D human model to a desired position and orientation and have an algorithm select the proper angles of the wrist, elbow, and shoulder joints.
- This usually requires that the figure move within reasonable anthropomorphic limits.



http://catiadoc.free.fr/online/cfyughbr_C2/cfyughbrikbhv.htm

Inverse kinematics

- Example: given **p**, determine θ_1 and θ_2

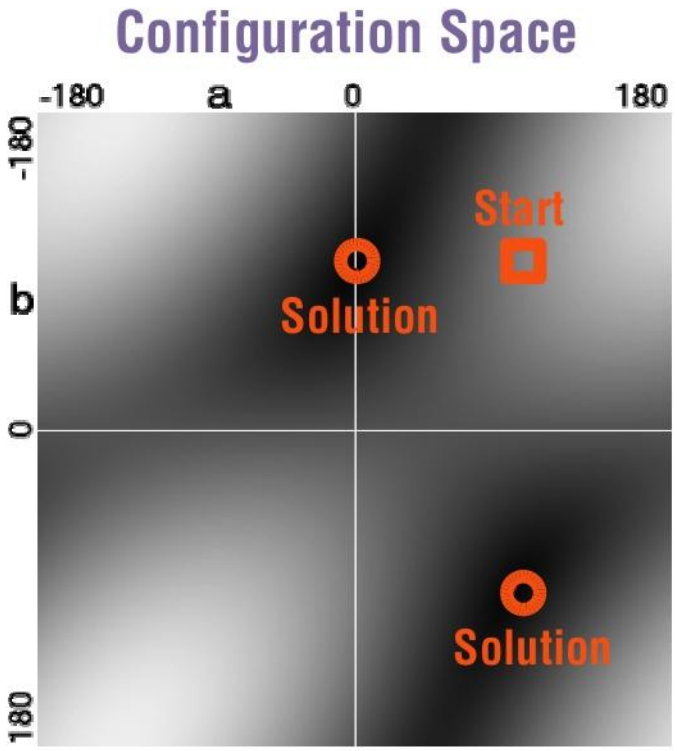


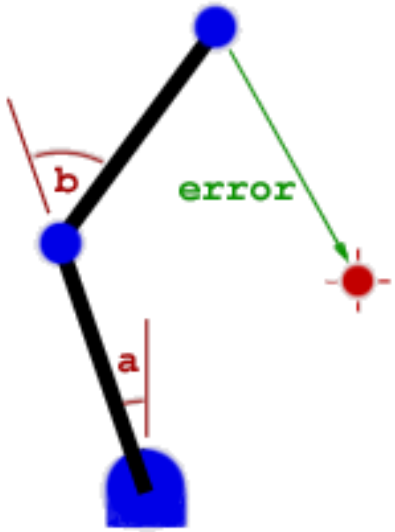
$$\theta_2 = \cos^{-1}\left(\frac{x^2 + y^2 - L_1^2 - L_2^2}{2L_1L_2}\right)$$

$$\theta_1 = \tan^{-1}\left(\frac{y(L_1 + L_2\cos\theta_2) - xL_2\sin\theta_2}{x(L_1 + L_2\cos\theta_2) + yL_2\sin\theta_2}\right)$$

Inverse kinematics

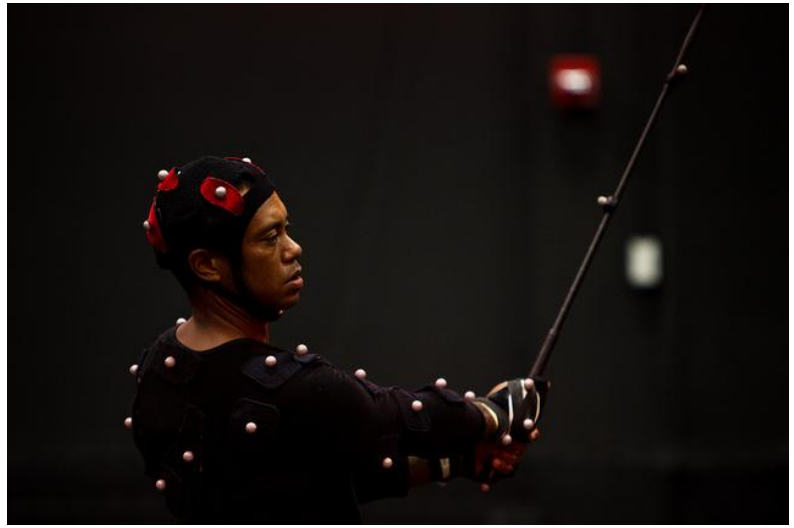
- Generally, the system of equations that must be solved is complex and non-linear. Analytic solutions are typically not possible.
- Instead, one can set up an optimisation problem that searches the configuration space of the joint angles for a minimum error configuration. Note there may be multiple global minima!





Motion capture

- Track data from real-world action
- Many techniques for data capture:
  - Optical – take video and extract motion
  - Magnetic/Radio – attach magnets, transponders and use sensors to get location
- Use data to drive an articulated model



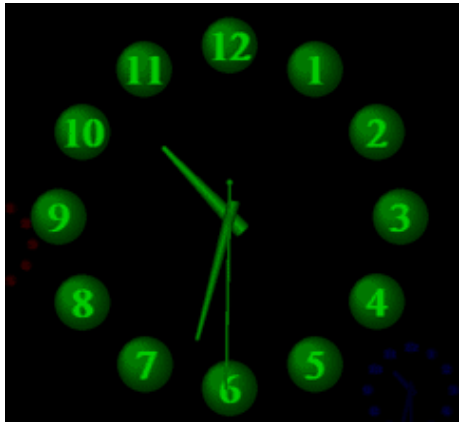
Electronic Arts: Tiger Woods PGA Tour 2013

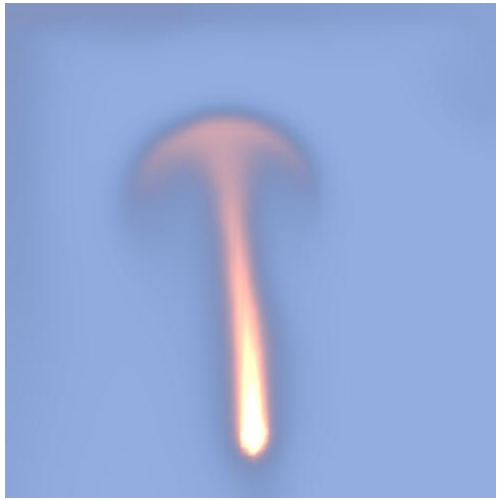
Motion capture

- Motion capture is one of the primary animation techniques for computer games
  - Gather lots of snippets of motion capture: e.g. dunk, dribble, pass
  - Arrange the snippets so that they can be pieced together smoothly: match end points to start points
  - At run time, figure out which pieces to play to have the character do the desired motion
- Challenges:
  - Data acquisition
  - Associating motion to surface geometry
  - Editing the data

Procedural animation

- Animation is described algorithmically
- Express animation as a function of a small number of parameters
- Example 1:
  - Clock with a second, minute, hour hands
  - Animate based on current time
- Example 2:
  - Fluid using computational fluid dynamics





Shahed Aziz, MSc Thesis 2012

Particle animation

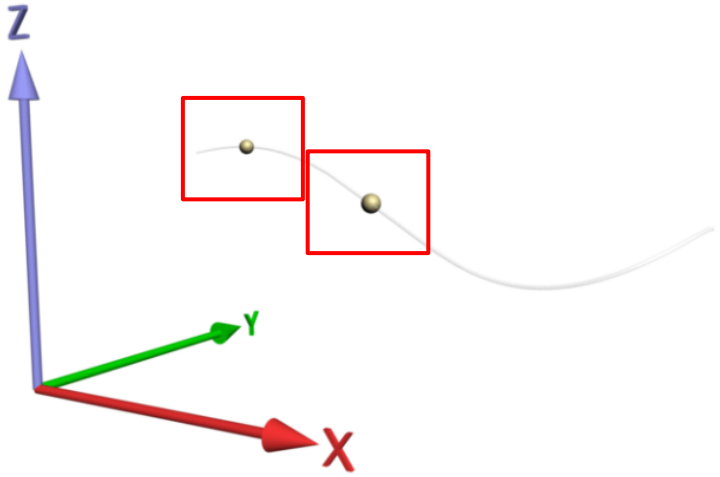
- Many of the stunning special effects in computer games are implemented using a particle animation
- A particle animation is simply a collection of particles (point masses) that move in 3D subject to various rules.
- Often, these rules are based on some sort of physical simulation

- *Particle systems* are collections of particles
  - Particles are usually rendered as a collection of billboarded sprites or graphic objects (e.g., spheres, cubes, points, etc.)
  - Used to simulate complex phenomena like fire, smoke, sparks, magic spells, clouds
  - An *emitter* is the source of particles in 3D space and sets the initial parameters of each particle (velocity, colour, etc.)
  - Particles move based on physical simulation
  - Particles change over time: **fade out**, changing their colour and alpha value

A particle’s properties

- At a minimum, a particle has
  - Life
  - Position
  - Velocity

- But it can have additional properties like
  - Acceleration
  - Image size
  - Image rotation angle
  - Image rotation angular velocity
  - Colour (include colour sequence)



GPU particles

- The position of a particle can be computed on the client side or on the GPU.
- Client-side particles:
  - Simulate the physical equations of the particles on the CPU; apply the appropriate transformations to the modelview matrix and render each particle.
- GPU particles:
  - Direct solution: The client sends the initial position, start time, and velocity of each particle. The equation of motion is evaluated in the vertex shader and the particle is rendered. Example: next slide.
  - Integration: The client sends the initial position, start time, and velocity of each particle. In the vertex shader, particles are moved one time step in a numerical integration.

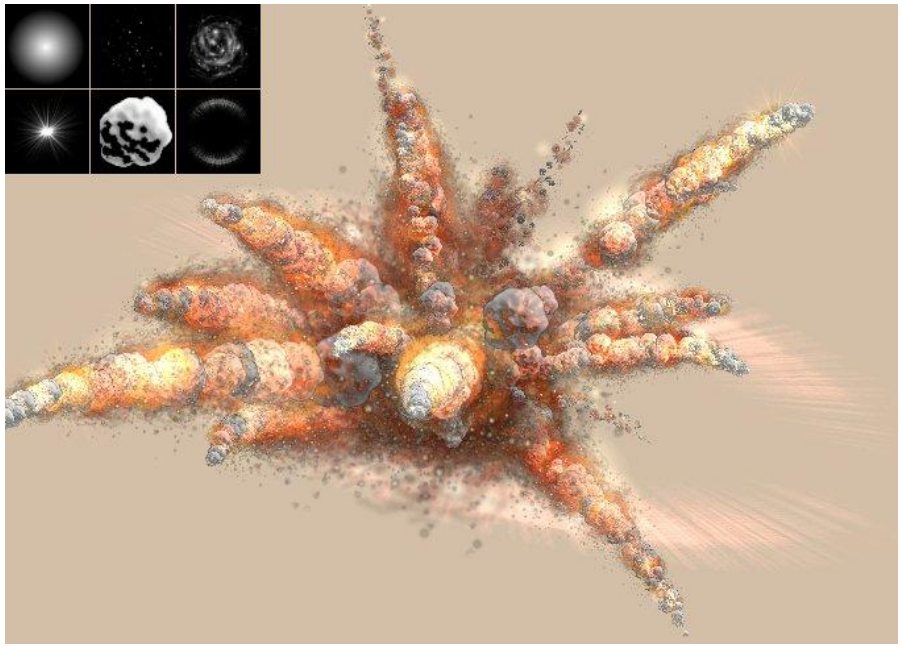
⇨ The OpenGL Cookbook Chapter 9 has examples of both approaches.

Example

From Cookbook, Chapter 9

Combinations of particles

- By combining multiple particles, one can achieve remarkable effects!

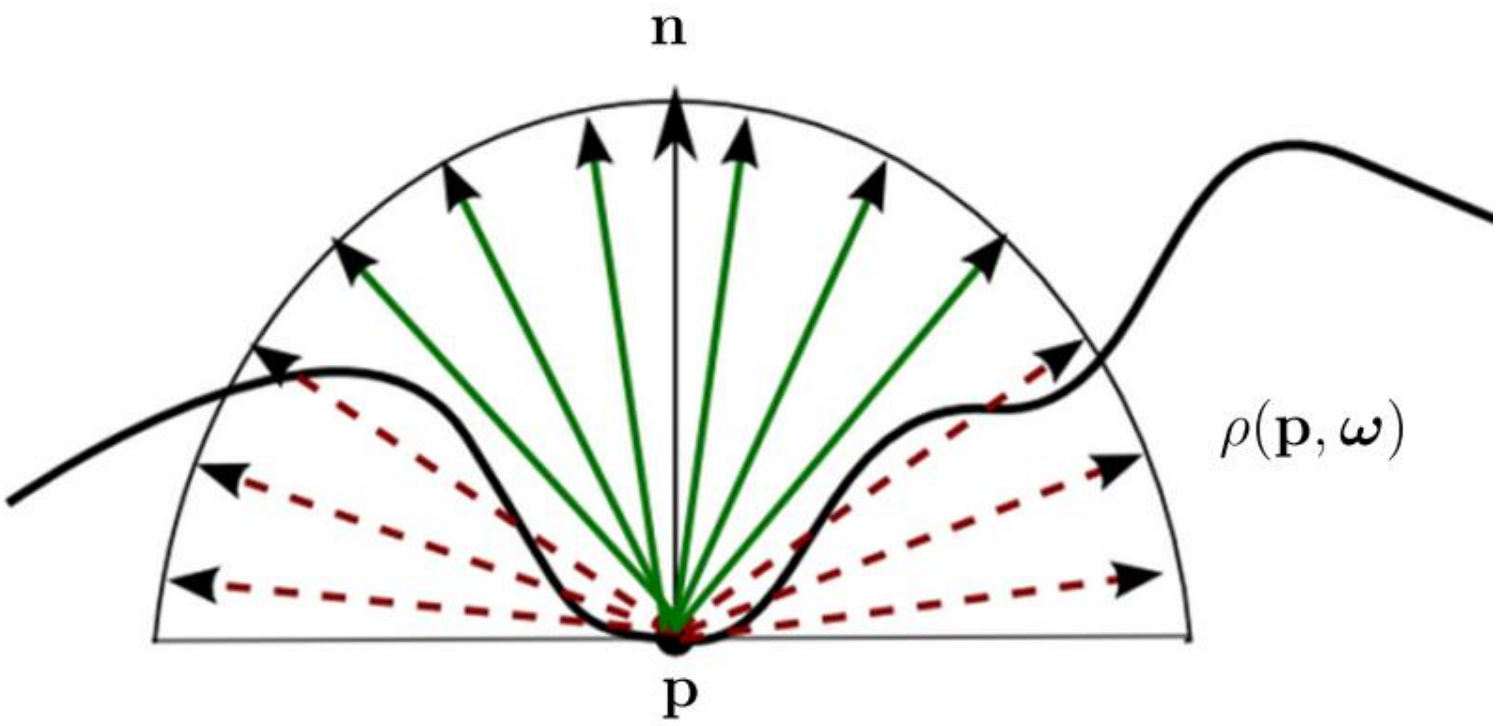


Ambient occlusion

- As discussed earlier, the ambient term in the lighting equation models light that reaches surfaces indirectly, for example light bouncing from nearby surfaces. This ambient term was the same for every point in the scene.
- However, in real life, due to creases or gaps between objects, less light will impinge on a surface, essentially *occluding* them from the light source.
- Ambient occlusion models this, by including an *accessibility* factor as a measure of how much ambient light reaches a point on a surface.

https://coderwall.com/p/uhik9g/three-js-baking

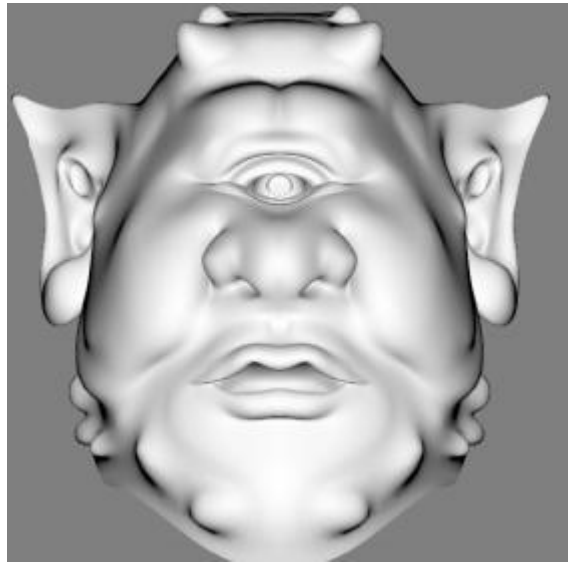
Accessibility

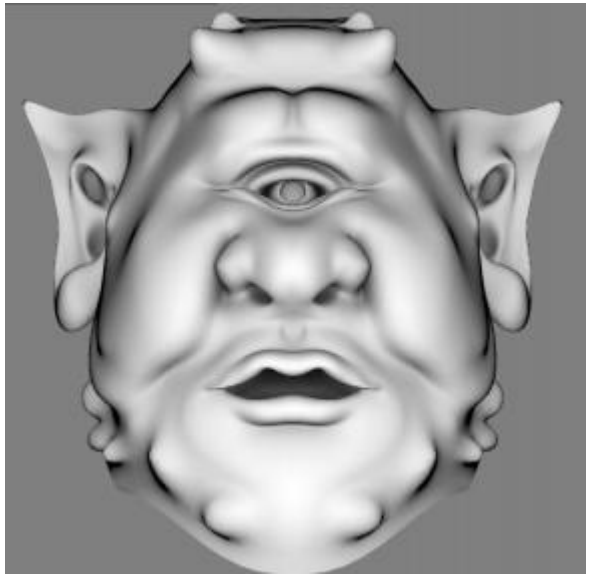


$$AO(\mathbf{p}) = \frac{1}{\pi} \int_{\Omega} \rho(\mathbf{p}, \boldsymbol{\omega}) \mathbf{n} \cdot \boldsymbol{\omega} d\boldsymbol{\omega}$$

Baked AO

- Some tools like Blender can pre-calculate (aka, *bake*) an ambient occlusion map and save it as a texture.
- When rendering, you can use the texture to reduce the brightness of fragments.
- Baking an AO map is efficient, but does not model the ambient occlusion resulting from other nearby objects and does not change with movement.
- An example of this is in the Cookbook.





Screen Space Ambient Occlusion (SSAO)

- Computes the accessibility factor in real-time, which must be done efficiently.
- This was first used in the game Crysis, using multi-pass rendering.
- Rather than evaluate the integral directly, a random (Monte-Carlo) sampling technique is used.
- Approach:
  - Pass 1: Render the scene to texture.
  - Pass 2: Randomly sample the depth buffer around the current point **p** (this can based on the normal if available). Count the points that are closer; these block the light coming to **p**. Encode occlusion as a value [0, 1].
  - Pass 3: Blur the output from Pass 2, and darken fragments based on the AO term.
- Tutorial: http://john-chapman-graphics.blogspot.co.uk/2013/01/ssao-tutorial.html and video: https://www.youtube.com/watch?v=-IFxjKT7MXA
- There are also details in the SuperBible, including shader code.

Example of SSAO

How fast should my game run?

- If the frame rate is *too low*, the illusion of motion may be impaired and can affect the user's ability to interact with the game.
- If the frame rate is *too high*, the frames may be updating faster than the screen refresh rate (often 60 FPS). In this case, some rendered frames are not displayed. Therefore, most graphics cards can limit the frame rate to the screen refresh rate (with v-sync).
- A reasonable target is a frame rate of 30 – 60 FPS. For example, Halo 4 is locked to 30 FPS (Halo 5 manages 60 with some loss of resolution).

Halo 4

Programming v-sync

- You can program OpenGL to perform v-syncing. This way, the graphics card will update the display when it refreshes the monitor.

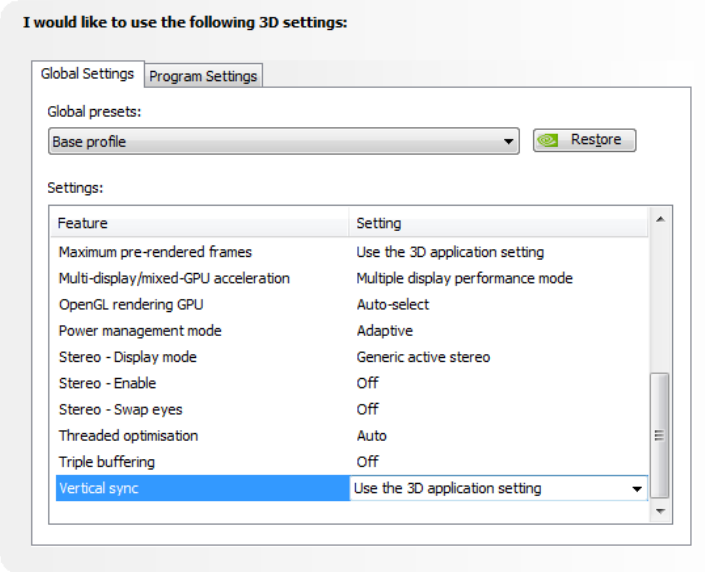
```
#include "./include/gl/wglew.h”
```

- V-sync on:

```
if( WGLEW_EXT_swap_control )
        wglSwapIntervalEXT(1);
```

- V-sync off:

```
if( WGLEW_EXT_swap_control )
        wglSwapIntervalEXT(0);
```



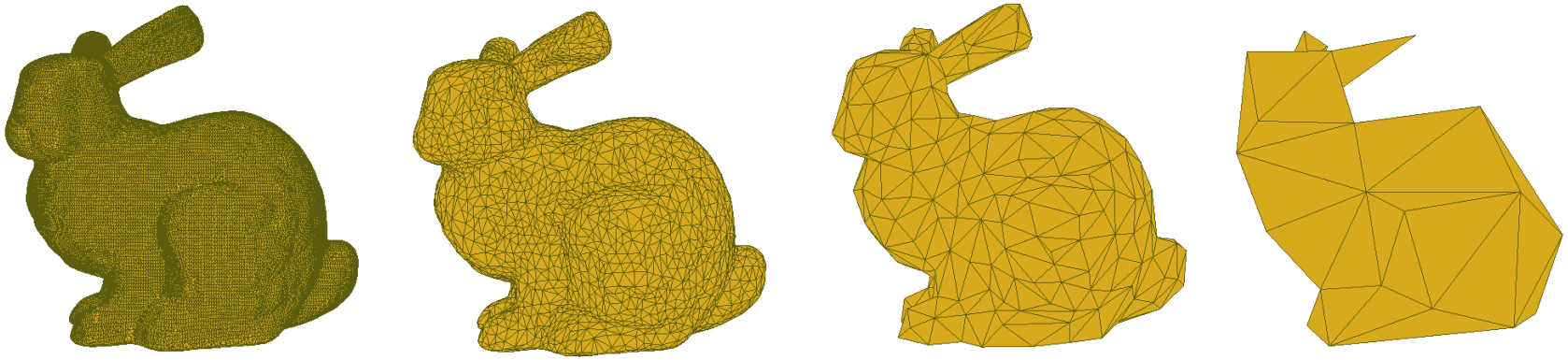
- Note: Graphics driver settings may over-ride calls in the OpenGL code. You may need to “Use the 3D application setting”

How do I speed up my graphics?

- *First and foremost*: Perform profiling to identify bottlenecks. The program NVidia Nsight Explorer is very helpful in this regard (if you have an NVidia GPU)
- Hardware
  - Try to make better use of the GPU
    - Use element arrays or triangle strips to reduce the number of VBO vertices
    - Use built-in GLSL functions
    - Eliminate unnecessary GLSL code
    - Consider moving fragment shader computations to vertex shader
    - Instanced rendering
  - Minimise state changes
    - Use VAOs
    - Use texture samplers
- Software
  - More efficient algorithms and data structures
    - Level of detail (next slide)
    - Reduce the number of passes in multi-pass algorithms
    - Reduce texture anisotropy (or turn it off)
  - Cull objects outside of view frustum (more in this lecture)
  - Enable backface culling

Decimation and LOD

In computer games, using meshes with many triangles can slow down rendering, even if using a VBO. It may be preferable to decimate (downsample) the mesh so that it has fewer triangles.



69,451 triangles | 6945 triangles | 694 triangles | 69 triangles

In a **level of detail (LOD)** system, one would use multiple versions of the surface. When it is far away from the camera, the low resolution version would be sufficient. Mipmapping is an example of LOD applied to textures. Tesselation shaders can be helpful for this purpose.

Mesh polygon count

Half-life 2: ~6000

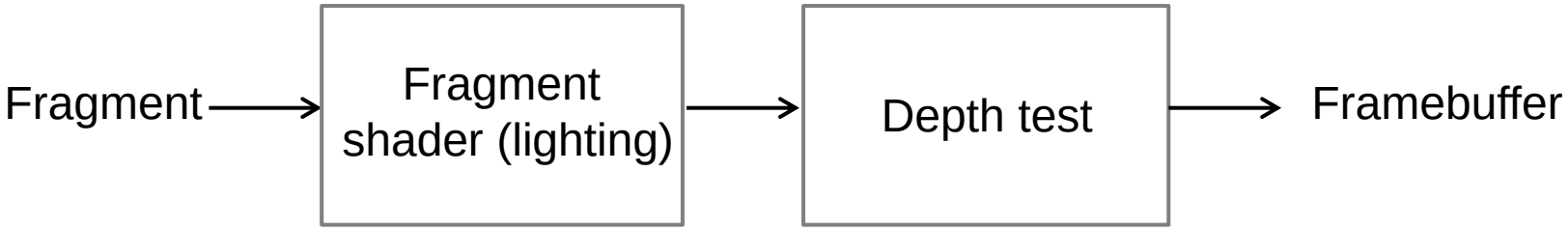
Virtua fighter 5: ~40,000

Gran turismo 5: ~200,000

http://forum.beyond3d.com/showthread.php?t=43975

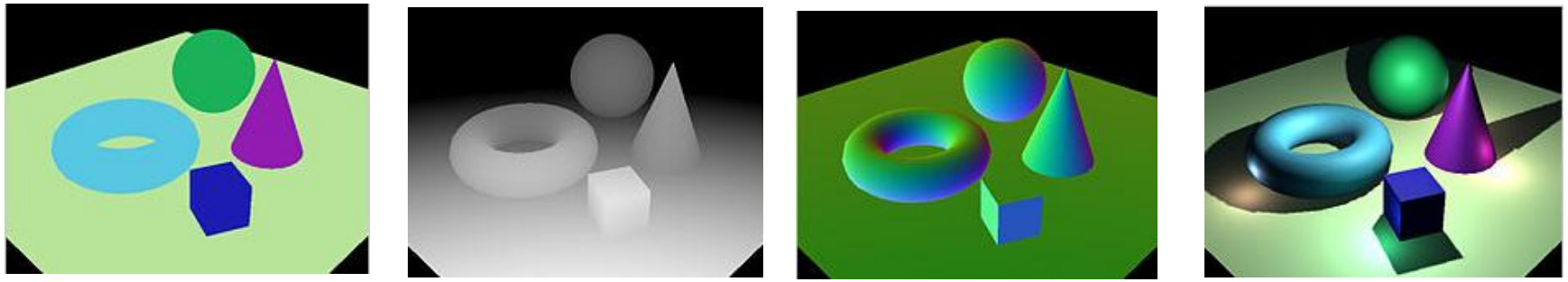
Forward rendering

- In the standard rendering pipeline (also called *forward rendering*), the lighting computation is done before the depth test.
- In per-fragment lighting, in the fragment shader, there is a loop over all the light sources in order to shade the fragment.
- Unfortunately, this means that the lighting equation is evaluated for occluded pixels that do not pass the depth test.
- For scenes with many lights this is undesirable.



Deferred rendering

- In *deferred rendering (aka deferred shading)*, the lighting computation is performed only for **visible** fragments, using multi-pass rendering.
  - In the first pass, the geometry is rendered to a geometry buffer, or “g-buffer,” providing colour, normals, depth, etc. for visible fragments.
  - In a second pass, lighting is computed only for the fragments in the g-buffer, hence the name *deferred* rendering.



Diffuse colour | Depth buffer | Normals | Final scene

G buffer

wikipedia

Pros and cons of deferred rendering

Pros

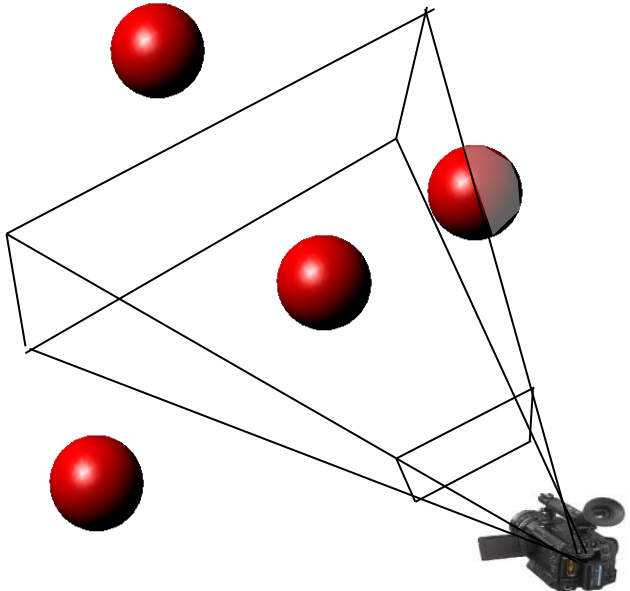
- The primary advantage of deferred rendering is that the lighting is only computed for fragments that are visible. Consider a scene with many lights:
  - In forward rendering, each fragment requires a loop over all light sources and for each, an evaluation of the reflectance model.
  - For many fragments, the lighting is computed, only to find that the fragment is not visible due to occlusion.
  - In this case it's better to only computing lighting for visible fragments.
- Deferred rendering provides better scalability as the number of light sources increases.

Cons

- The primary disadvantage of deferred rendering is the additional storage for the g-buffer. Everything used in lighting must be stored there.
- This becomes a difficulty when different materials are required for different objects in the scene.

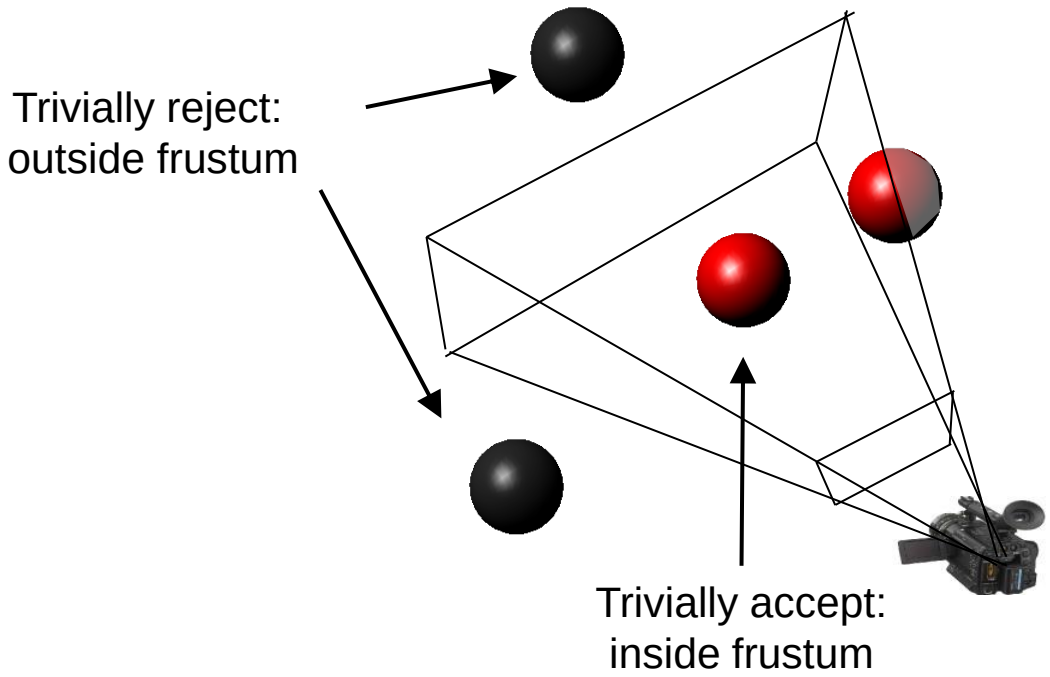
Culling and clipping

- When it comes to framerate, what is *not* being rendered is arguably as important as what *is* being rendered.
- Despite state-of-the-art hardware, it may be computationally expensive to send many primitives to be transformed, textured, and shaded only to find the primitive is not visible, possibly not even in the view frustum. This is especially true in today's multi-pass rendering techniques.
- Any primitive completely outside the view frustum does not need to be rendered (it can be *culled*).
- For primitives partially in, only the visible part needs to be drawn (it is *clipped*)



Trivial acceptance tests

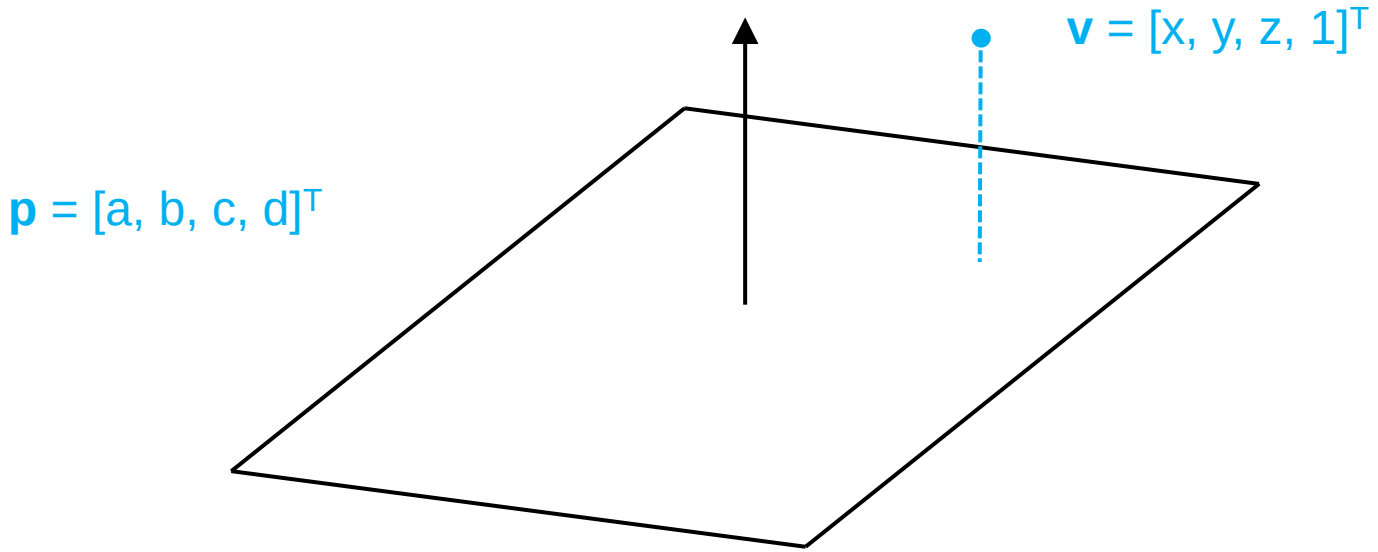
- View volume in/out tests can be easily performed. These tests are also known as *trivial acceptance tests*.
- Each side of the frustum defines a plane; any object or polygon on the “wrong” side of any plane (in the negative half space not including the view volume) can be skipped.



How many planar sides does the frustum have?

Point / plane test

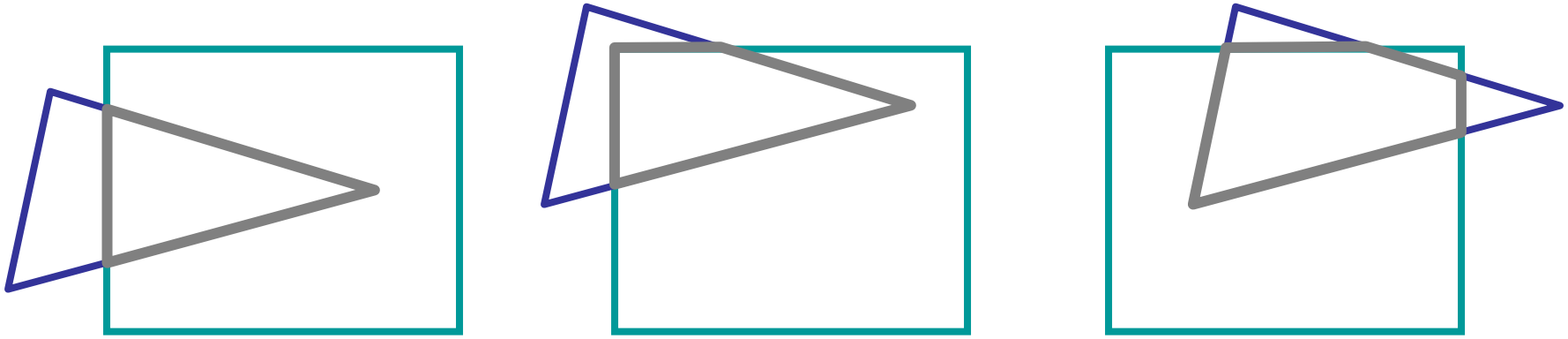
- Given a point **v** and the equation of a plane **p**, how can one tell if a point is in the positive or negative half-space of a plane?



⇨ Simply look at the sign of $\mathbf{p} \bullet \mathbf{v}$

- Simple example: consider the xz plane, $\mathbf{p} = [0, 1, 0, 0]^T$. A point **v** is above the plane if its y value is positive, in which case $\mathbf{p} \bullet \mathbf{v} > 0$.

Non-trivial tests (2D)



triangle □ triangle triangle □ quad triangle □ pentagon

There are numerous clipping algorithms to handle such cases. A well known approach is *Cohen-Sutherland clipping*, which can be used for 2D or 3D clipping.

Doing one's own trivial acceptance tests

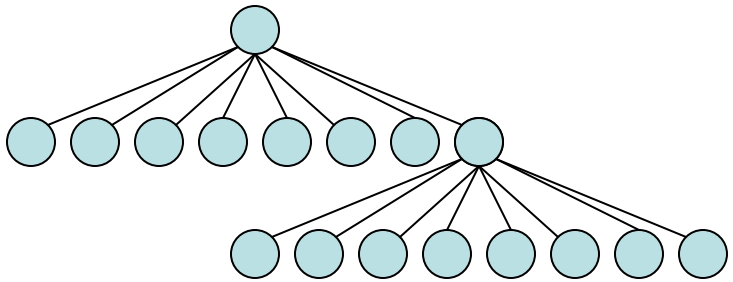
- When rendering primitives, OpenGL does not have knowledge of full 3D objects – it only sees one primitive at a time.
- For this reason, it may be more efficient if the programmer organises objects and performs his/her own culling (particularly trivial acceptance tests) before sending objects to OpenGL.
- However, for primitives or objects partially in the view volume, it may be more convenient to leave these for OpenGL to clip.
- A simple approach would be to have a bounding box around each 3D object. If all eight vertices of the object's bounding box are in the *same* negative half-space of a view frustum plane, the entire object can be skipped.

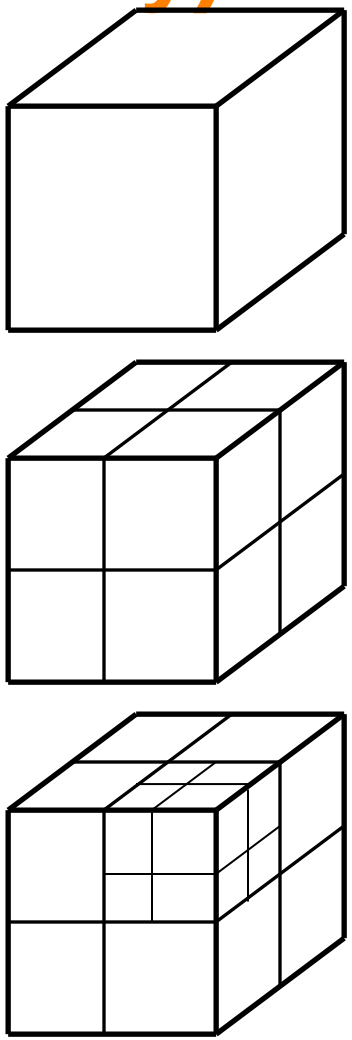
Spatial organisation

- While a bounding box test can be useful, often computer games may have more advanced spatial organisation algorithms to limit the rendering to the visible portion of the scene.
- The are numerous approaches, including BSP trees, portals, hierarchical occlusion maps, etc.
- An *octree* is a hierarchical tree structure used to partition a 3D space. Each octree node has eight children (hence the name “octree”).

Octree (diagrammatically)

- Represent the 3D volume with a bounding box.
- Subdivide by half in along each axis to give 8 child voxels.
- Repeatedly subdivide each child voxel the same way if required.



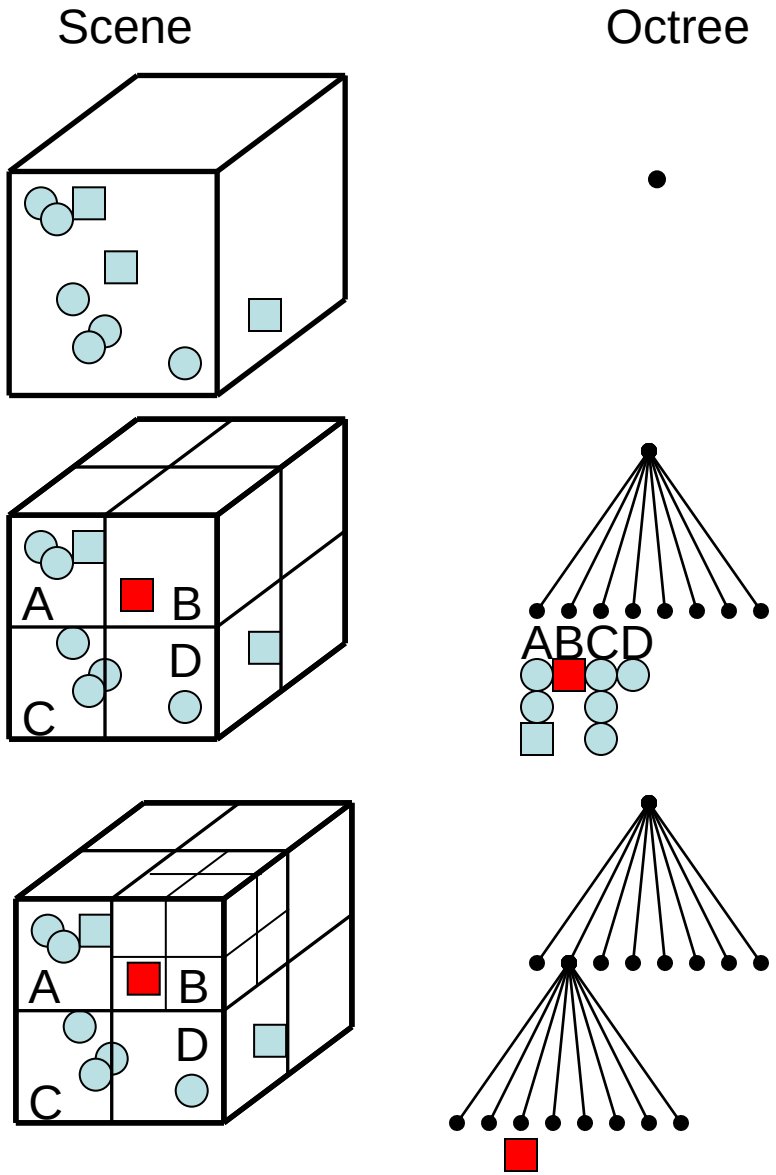


Octree construction example

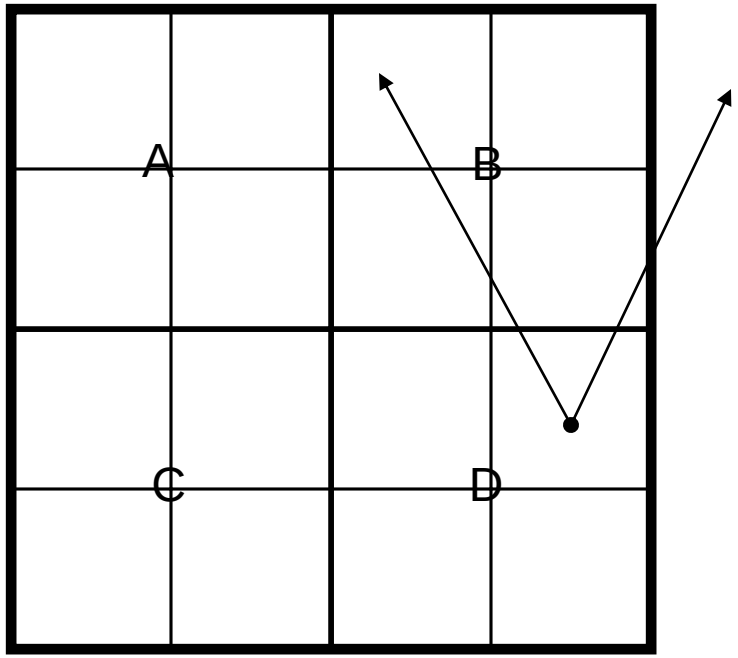
1. Put a bounding box around all objects. (this is the root of the octree).

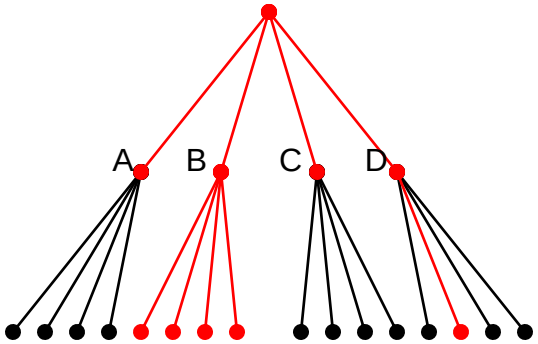
2. Subdivide each node into 8 voxels. These will be the children of the node. Put each object in the respective node.

3. For each node, subdivide again recursively, until MAX_LEVEL has been reached or there is no object left in the node.
*The node B is shown.



Visibility queries with an octree





This example shows a quad-tree, a 2D version of an octree, for simplicity.
The camera and view frustum are shown.
Only the red nodes need to be tested for visibility.
All the objects contained in the leaf red nodes need to be tested for visibility.

Object oriented programming with OpenGL

- OpenGL has no notion of object oriented programming. It is merely an interface to the graphics hardware.
- As graphics programmers, we can build 3D objects in code using object-oriented languages/techniques.
- In C++, one can
  - Create various 3D objects
  - Hide implementation details using public, private, protected members variables and methods
  - Apply operator overloading for suitable functions
  - Organise the scene as a collection of 3D objects

Cube object

- Suppose that we want to create a simple cube object

```
cube.initialise();
```

- We also need a way of displaying the cube

```
cube.render();
```

- We can set properties as part of the class, for example

```
cube.translate(1.0, 0.0, 0.0);
cube.rotate(theta, 1.0, 0.0, 0.0);
cube.setcolor(1.0, 0.0, 0.0);
```

Other objects

- Other objects also have geometric aspects
  - Cameras
  - Light sources
- We can have non-geometric objects too
  - Materials
  - Colours
  - Transformations (matrices)

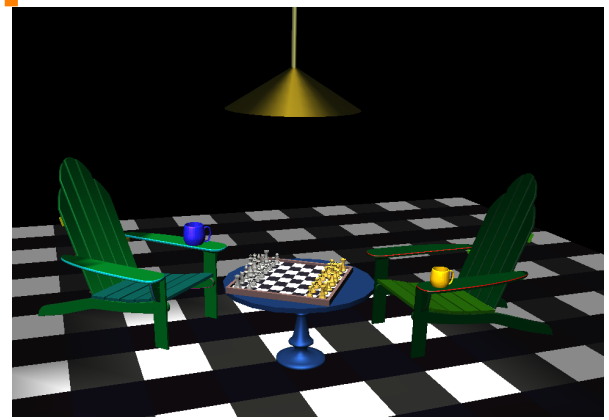
Graphical objects

```
class GraphicalObject
{
public:
   GraphicalObject() {}
   virtual ~GraphicalObject();
   virtual void Render();
};
```

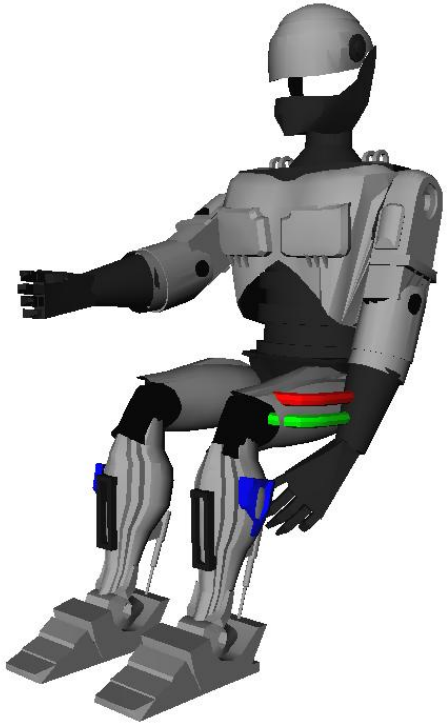
A triangle

```
class Triangle : public GraphicalObject
{
public:
   Triangle();
   virtual ~Triangle();
   virtual void Render();
   void AddVertex3d(glm::vec3 v);
   void AddVertex3dv(glm::vec3 *p);
   void AddVertices(glm::vec3 a, glm::vec3 b, glm::vec3 c);
private:
   glm::vec3 v0;
   glm::vec3 v1;
   glm::vec3 v2;
};
```

Scene graph



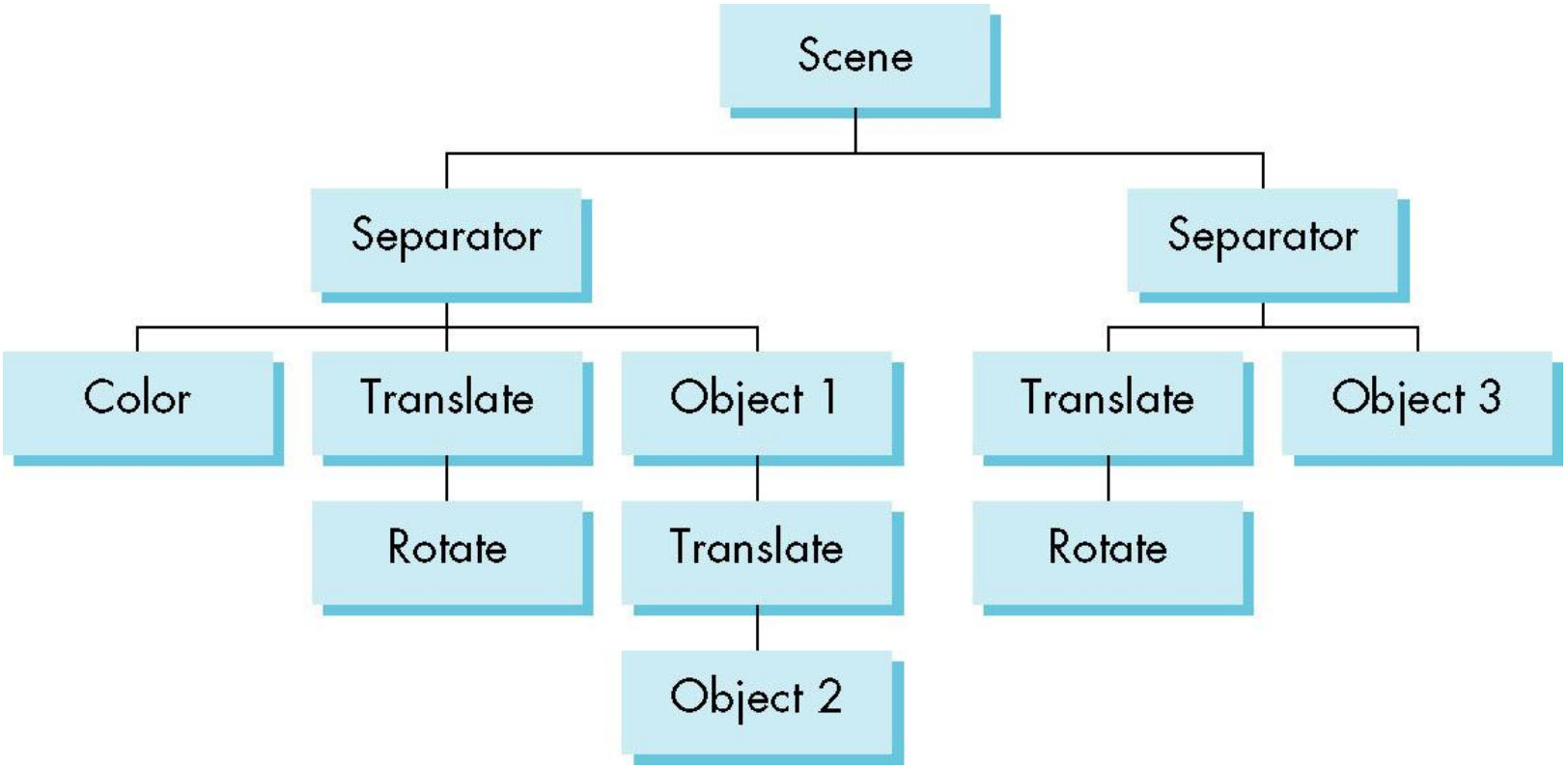
- Convenient data structure for scene representation
  - Transformations
  - Materials, colour
  - Multiple instances
- Basic idea: Tree
- Useful for manipulation/animation
  - Especially for articulated figures



Node types:

Shape: 3D geometric objects
Transform: effect current transformation
Property: appearance, texture, etc.
Group: collection of sub-graphs

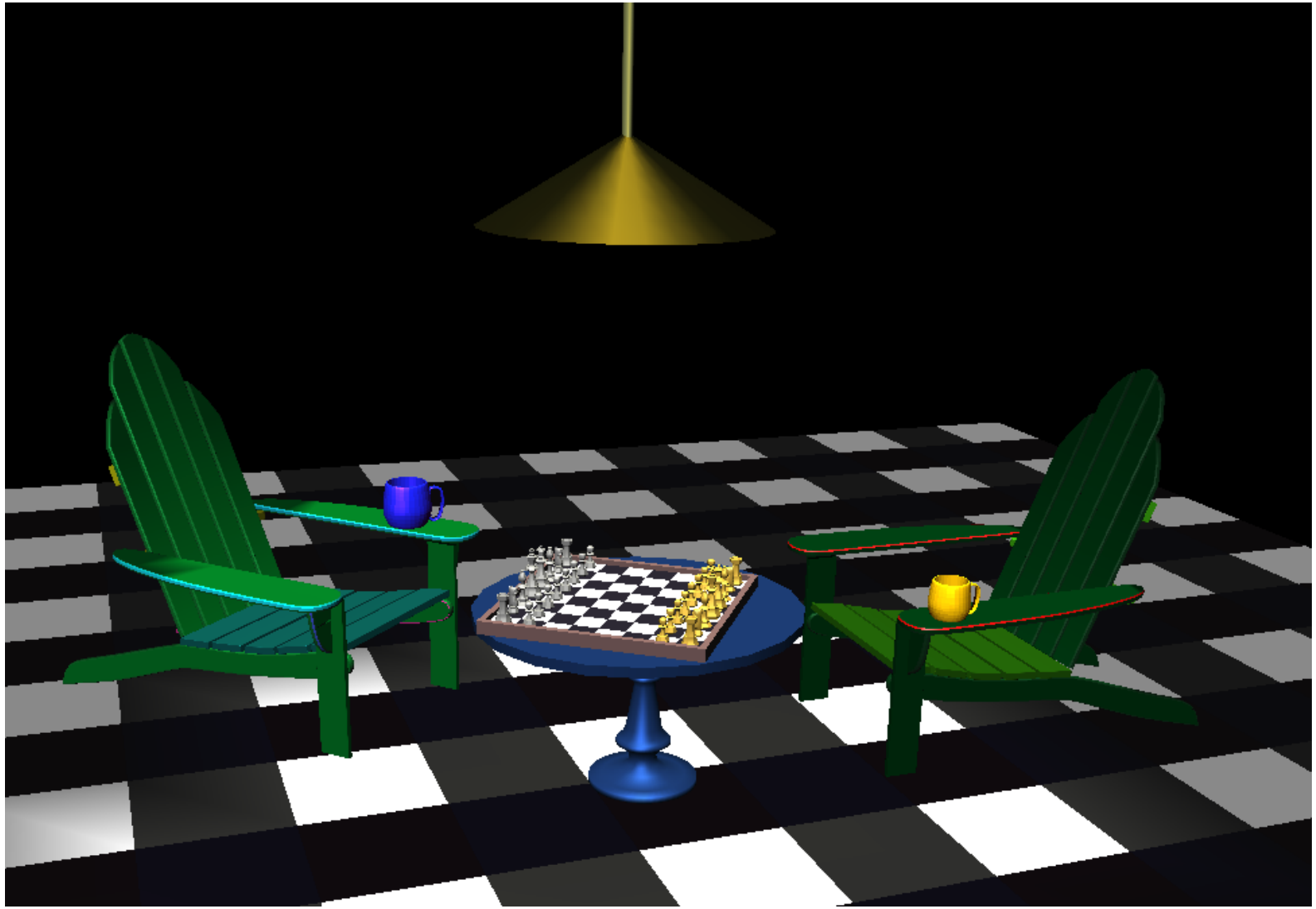
Scene graph example

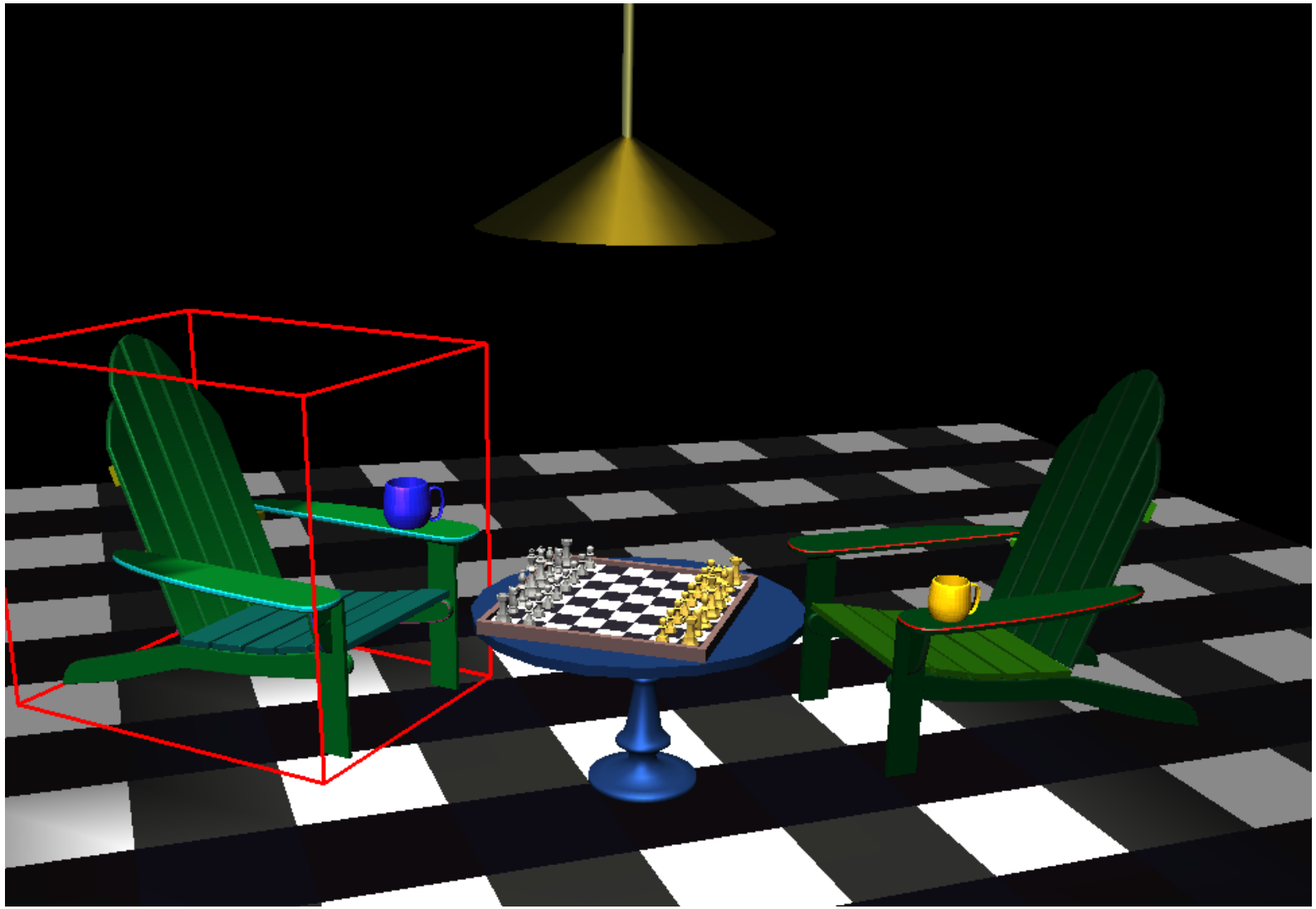


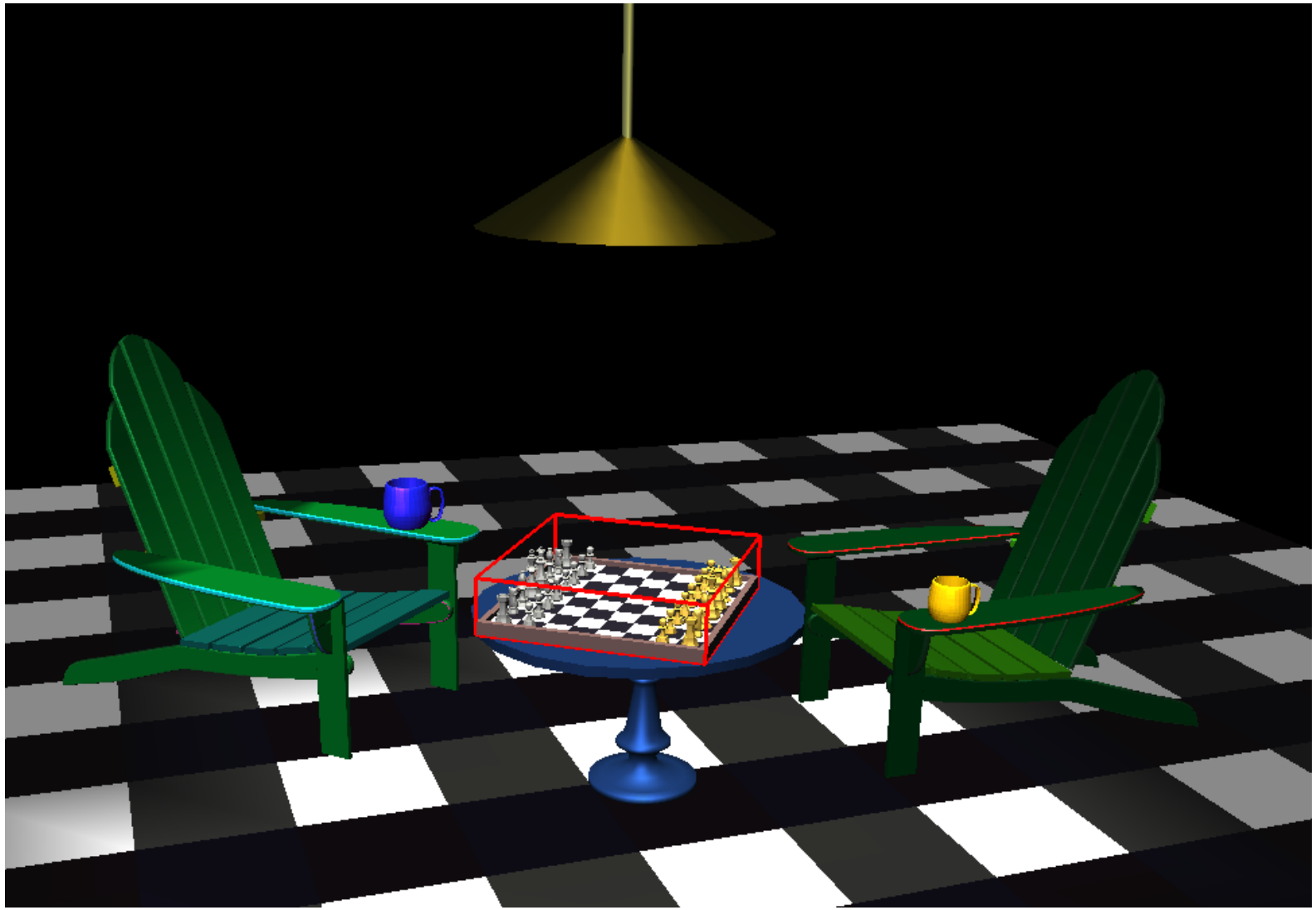
Scene graph representations

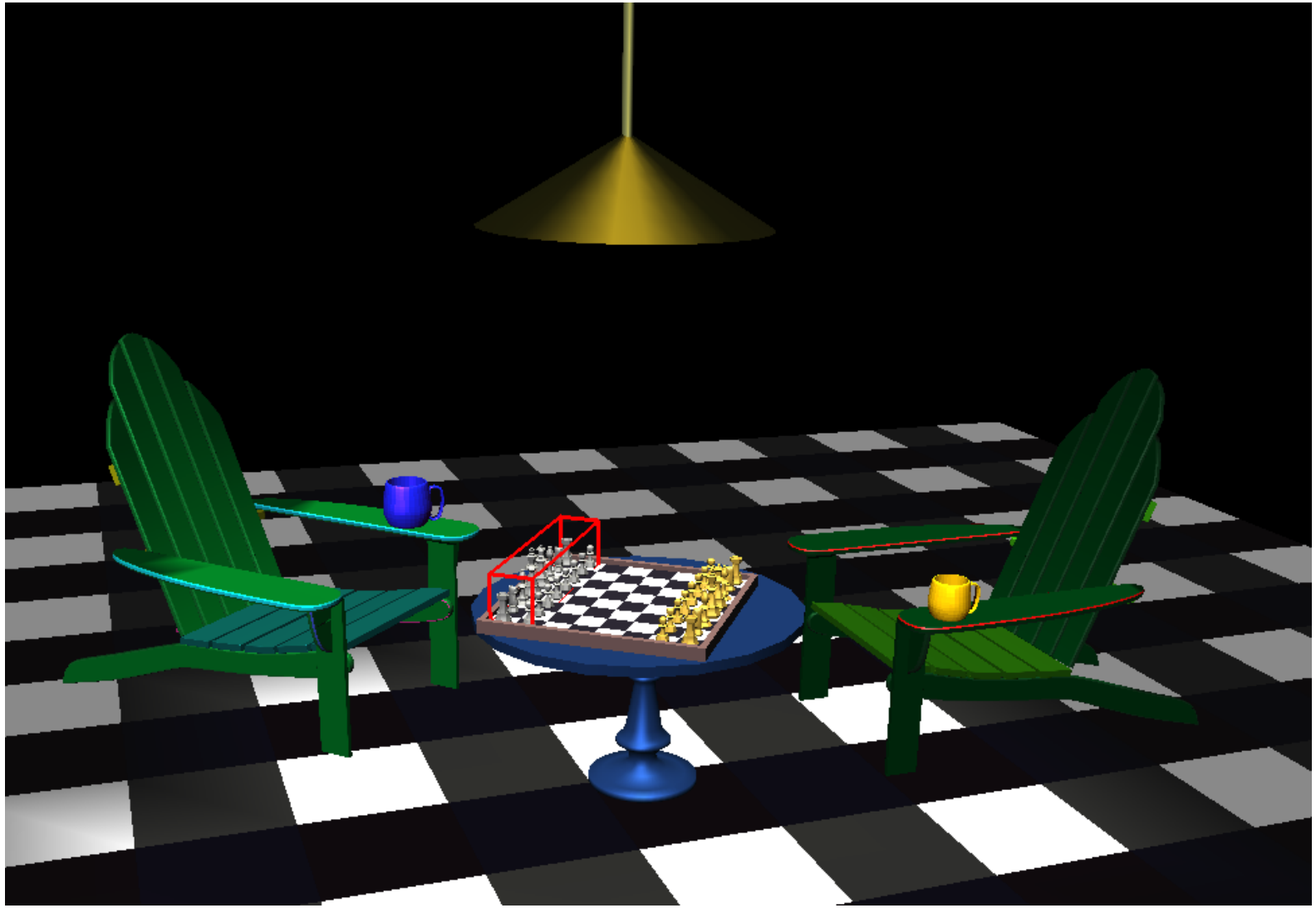
- The OpenGL state is updated during traversal of the graph (top to bottom, left to right)
- The effect of a parent applied to all its child nodes; an operation performed on a group automatically propagates its effect to all of its members.
- In many programs, associating a geometrical transformation matrix at each group level and concatenating such matrices together is an efficient and natural way to process such operations (recall lab 2)
- A common feature, for instance, is the ability to group related shapes/objects into a compound object that can then be moved, transformed, selected, etc. as easily as a single object.

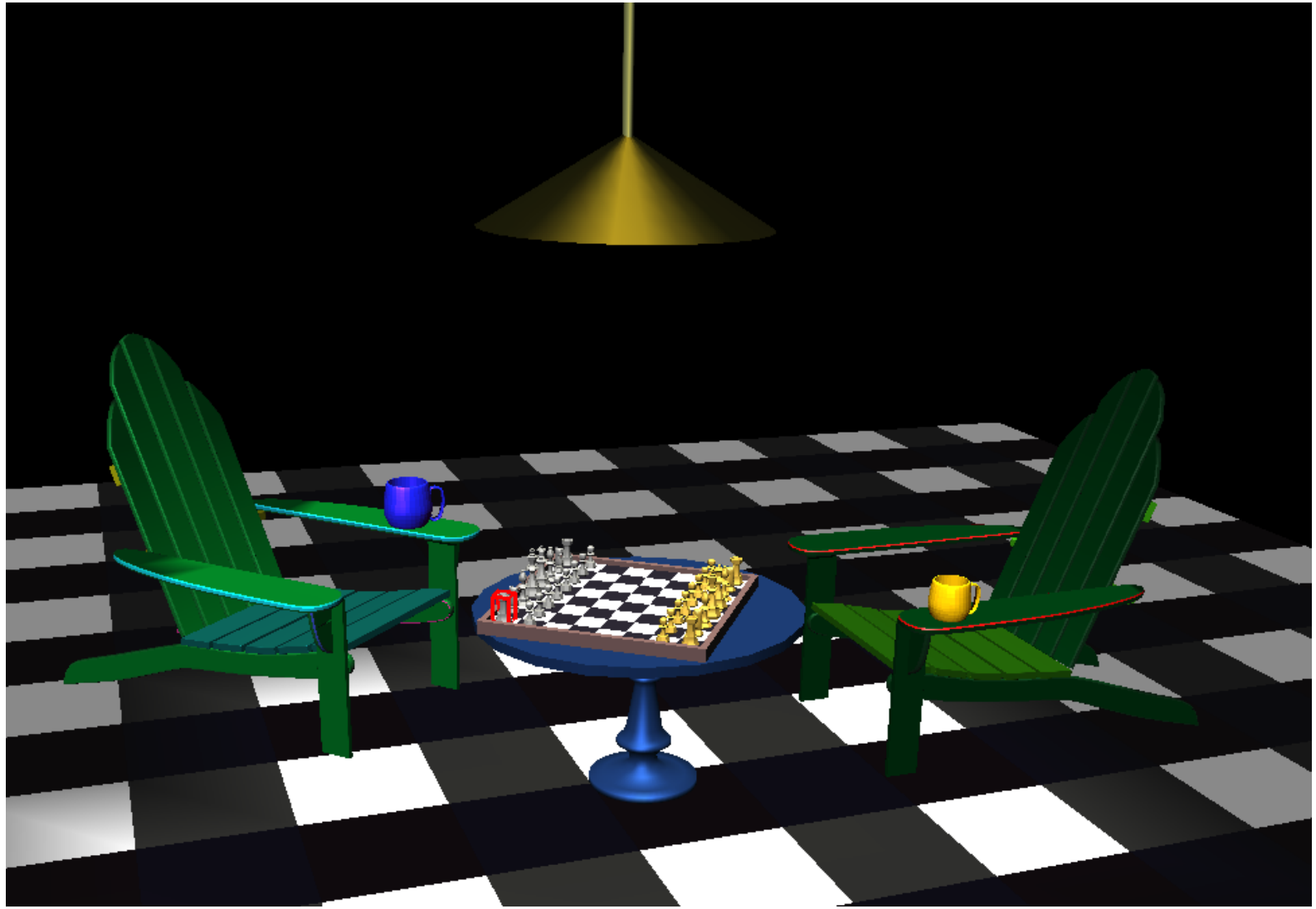
Example











Module Evaluation Survey

- A formal way to gather your feedback to improve the student experience
- More information on the Student Hub
- Access all open surveys on the Student Survey Portal or via the QR code (you will also be emailed a link)